new idea

SELECT WITH FUNCTIONS

Select with Modification

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
myuniversitydb=# SELECT id, name, salary*2
myuniversitydb-# FROM teacher;
  id   |    name    | ?column?
-------+------------+----------
 10101 | Srinivasan | 130000.00
 12121 | Wu         | 180000.00
 15151 | Mozart     |  80000.00
 22222 | Einstein   | 190000.00
 32343 | El Said    | 120000.00
 33456 | Gold       | 174000.00
 45565 | Katz       | 150000.00
 58583 | Califieri  | 124000.00
 76543 | Singh      | 160000.00
 76766 | Crick      | 144000.00
 83821 | Brandt     | 184000.00
 98345 | Kim        | 160000.00
(12 rows)
```

```
SELECT id, name, salary * 2
FROM teacher ;
```



id	*name*	*?column?*
10101	Srinivasan	130000.00
12121	Wu	180000.00
15151	Mozart	80000.00
22222	Einstein	190000.00
32343	El Said	120000.00
33456	Gold	174000.00
45565	Katz	150000.00
58583	Califieri	124000.00
76543	Singh	160000.00
76766	Crick	144000.00
83821	Brandt	184000.00
98345	Kim	160000.00

Select with Modification

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT CONCAT(name, 'InUniversity')
FROM teacher ;
```


concat
SrinivasanInUniversity
WuInUniversity
MozartInUniversity
EinsteinInUniversity
El SaidInUniversity
GoldInUniversity
KatzInUniversity
CalifieriInUniversity
SinghInUniversity
CrickInUniversity
BrandtInUniversity
KimInUniversity

```
myuniversitydb-# SELECT CONCAT(name, 'InUniversity')
                  FROM teacher ;
         concat
-----------------------
 SrinivasanInUniversity
 WuInUniversity
 MozartInUniversity
 EinsteinInUniversity
 El SaidInUniversity
 GoldInUniversity
 KatzInUniversity
 CalifieriInUniversity
 SinghInUniversity
 CrickInUniversity
 BrandtInUniversity
 KimInUniversity
(12 rows)
```

Select with Modification

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name || 'InUniversity'
FROM teacher ;
```



?column?
SrinivasanInUniversity
WuInUniversity
MozartInUniversity
EinsteinInUniversity
El SaidInUniversity
GoldInUniversity
KatzInUniversity
CalifieriInUniversity
SinghInUniversity
CrickInUniversity
BrandtInUniversity
KimInUniversity

```
myuniversitydb=# SELECT name || 'InUniversity'
                  FROM teacher ;
        ?column?
-----------------------
 SrinivasanInUniversity
 WuInUniversity
 MozartInUniversity
 EinsteinInUniversity
 El SaidInUniversity
 GoldInUniversity
 KatzInUniversity
 CalifieriInUniversity
 SinghInUniversity
 CrickInUniversity
 BrandtInUniversity
 KimInUniversity
(12 rows)
```

new idea

AS

recall

Products with Renaming

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

corelation name or table alias

```
SELECT *
FROM department AS A, department AS B;
```

dept_name	*building*	*budget*	*dept_name*	*building*	*budget*
Biology	Watson	90000.00	Biology	Watson	90000.00
Biology	Watson	90000.00	Comp. Sci.	Taylor	100000.00
Biology	Watson	90000.00	Elec. Eng.	Taylor	85000.00
Biology	Watson	90000.00	Finance	Painter	120000.00
Biology	Watson	90000.00	History	Painter	50000.00
Biology	Watson	90000.00	Music	Packard	80000.00
Biology	Watson	90000.00	Physics	Watson	70000.00
Comp. Sci.	Taylor	100000.00	Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00	Comp. Sci.	Taylor	100000.00
		100000.00	Elec. Eng.	Taylor	85000.00
		100000.00	Finance	Painter	120000.00
		100000.00	History	Painter	50000.00
		100000.00	Music	Packard	80000.00
		100000.00	Physics	Watson	70000.00
Elec. Eng.	Taylor	85000.00	Biology	Watson	90000.00
Elec. Eng.	Taylor	85000.00	Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00	Elec. Eng.	Taylor	85000.00
Elec. Eng.	Taylor	85000.00	Finance	Painter	120000.00
Elec. Eng.	Taylor	85000.00	History	Painter	50000.00
Elec. Eng.	Taylor	85000.00	Music	Packard	80000.00
Elec. Eng.	Taylor	85000.00	Physics	Watson	70000.00
Finance	Painter	120000.00	Biology	Watson	90000.00
Finance	Painter	120000.00	Comp. Sci.	Taylor	100000.00
Finance	Painter	120000.00	Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00	Finance	Painter	120000.00
Finance	Painter	120000.00	History	Painter	50000.00
Finance	Painter	120000.00	Music	Packard	80000.00
Finance	Painter	120000.00	Physics	Watson	70000.00
History	Painter	50000.00	Biology	Watson	90000.00
History	Painter	50000.00	Comp. Sci.	Taylor	100000.00
History	Painter	50000.00	Elec. Eng.	Taylor	85000.00
History	Painter	50000.00	Finance	Painter	120000.00
History	Painter	50000.00	History	Painter	50000.00
History	Painter	50000.00	Music	Packard	80000.00
History	Painter	50000.00	Physics	Watson	70000.00
Music	Packard	80000.00	Biology	Watson	90000.00

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

SELECT WITH RENAMING

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT id, name, salary * 1.1 AS newsalary
FROM teacher ;
```

id	*name*	*newsalary*
10101	Srinivasan	71500.000
12121	Wu	99000.000
15151	Mozart	44000.000
22222	Einstein	104500.000
32343	El Said	66000.000
33456	Gold	95700.000
45565	Katz	82500.000
58583	Califieri	68200.000
76543	Singh	88000.000
76766	Crick	79200.000
83821	Brandt	101200.000
98345	Kim	88000.000

```
myuniversitycb # SELECT id, name, salary * 1.1 AS newsalary
FROM teacher ;
  id   |     name     | newsalary
       |              |
 10101 | Srinivasan   |  71500.000
 12121 | Wu           |  99000.000
 15151 | Mozart       |  44000.000
 22222 | Einstein     | 104500.000
 32343 | El Said      |  66000.000
 33456 | Gold         |  95700.000
 45565 | Katz         |  82500.000
 58583 | Califieri    |  68200.000
 76543 | Singh        |  88000.000
 76766 | Crick        |  79200.000
 83821 | Brandt       | 101200.000
 98345 | Kim          |  88000.000
(12 rows)
```

RENAMING

We can rename both

attribute names **and** table names

using the keyword **AS**

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name AS mymodifiedname, id
FROM teacher ;
```

```
myuniversitydb=# SELECT name AS mymodifiedname, id
                 FROM teacher ;
 mymodifiedname |  id
                |
 Srinivasan     | 10101
 Wu             | 12121
 Mozart         | 15151
 Einstein       | 22222
 El Said        | 32343
 Gold           | 33456
 Katz           | 45565
 Califieri      | 58583
 Singh          | 76543
 Crick          | 76766
 Brandt         | 83821
 Kim            | 98345
(12 rows)
```

```
SELECT T.name, T.salary
FROM teacher AS T
WHERE T.salary >= 80000;
```

```
myuniversitydb=# SELECT T.name, T.salary
                 FROM teacher AS T
                 WHERE T.salary >= 80000;
   name   |  salary
----------+----------
 Wu       | 90000.00
 Einstein | 95000.00
 Gold     | 87000.00
 Singh    | 80000.00
 Brandt   | 92000.00
 Kim      | 80000.00
(6 rows)
```

RENAMING

We can rename both

attribute names **and** table names

using the keyword **AS**

```
SELECT name AS mymodifiedname, id
FROM teacher ;
```

```
myuniversitydb=# SELECT name AS mymodifiedname, id
                 FROM teacher ;
 mymodifiedname |  id
----------------+-------
 Srinivasan     | 10101
 Wu             | 12121
 Mozart         | 15151
 Einstein       | 22222
 El Said        | 32343
 Gold           | 33456
 Katz           | 45565
 Califieri
 Singh
 Crick
 Brandt
 Kim
(12 rows)
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT T.name, T.salary
FROM teacher AS T
WHERE T.salary >= 80000;
```

```
myuniversitydb=# SELECT T.name, T.salary
                 FROM teacher AS T
                 WHERE T.salary >= 80000;
  name   |  salary
---------+----------
 Wu      | 90000.00
 Einstein| 95000.00
 Gold    | 87000.00
 Singh   | 80000.00
 Brandt  | 92000.00
 Kim     | 80000.00
(6 rows)
```

when a table is renamed in **FROM**

→ its new name can be used in **SELECT** and **WHERE**

RENAMING

```
SELECT T.name, salary, dept_name
FROM teacher AS T
WHERE salary >= 80000;
```

```
myuniversitydb=# SELECT T.name, salary, dept name
                 FROM teacher AS T
                 WHERE salary >= 80000;
   name   |  salary  | dept_name
----------+----------+------------
 Wu       | 90000.00 | Finance
 Einstein | 95000.00 | Physics
 Gold     | 87000.00 | Physics
 Singh    | 80000.00 | Finance
 Brandt   | 92000.00 | Comp. Sci.
 Kim      | 80000.00 | Elec. Eng.
(6 rows)
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT T.name, T.salary
FROM teacher AS T
WHERE T.salary >= 80000;
```

```
myuniversitydb=# SELECT T.name, T.salary
                 FROM teacher AS T
                 WHERE T.salary >= 80000;
  name   |  salary
---------+----------
 Wu      | 90000.00
 Einstein| 95000.00
 Gold    | 87000.00
 Singh   | 80000.00
 Brandt  | 92000.00
 Kim     | 80000.00
(6 rows)
```

when a table is renamed in **FROM**

use with **or without** table name

...as long as no ambiguity

```
myuniversitydb=# SELECT name
myuniversitydb-# FROM teacher AS T, teacher;
ERROR:  column reference "name" is ambiguous
LINE 1: SELECT name
               ^
```

Renaming

when a table is renamed in **FROM**

use with **or** without table name
...as long as no ambiguity

```
myuniversitydb=# SELECT name
myuniversitydb-# FROM teacher AS T, teacher;
ERROR:  column reference "name" is ambiguous
LINE 1: SELECT name
               ^
```

```
myuniversitydb=# SELECT distinct T.name
myuniversitydb-# FROM teacher AS T, teacher;
    name
------------
 Einstein
 Katz
 Singh
 Kim
 Brandt
 El Said
 Wu
 Srinivasan
 Crick
 Gold
 Califieri
 Mozart
(12 rows)
```

Renaming

when a table is renamed in **FROM**

use with **or** without table name
...as long as no ambiguity

```
myuniversitydb=# SELECT name
myuniversitydb-# FROM teacher AS T, teacher;
ERROR:  column reference "name" is ambiguous
LINE 1: SELECT name
               ^
```

```
myuniversitydb=# SELECT distinct teacher.name
myuniversitydb-# FROM teacher AS T, teacher;
    name
------------
 Einstein
 Katz
 Singh
 Kim
 Brandt
 El Said
 Wu
 Srinivasan
 Crick
 Gold
 Califieri
 Mozart
(12 rows)
```

Select with Renaming

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT CONCAT(name, 'InUniversity') AS anewname
FROM teacher ;
```



anewname
SrinivasanInUniversity
WuInUniversity
MozartInUniversity
EinsteinInUniversity
El SaidInUniversity
GoldInUniversity
KatzInUniversity
CalifieriInUniversity
SinghInUniversity
CrickInUniversity
BrandtInUniversity
KimInUniversity

Select with Renaming

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Find all course ids taught in both Spring 2010 and Fall 2009.

```
SELECT course_id
FROM section as A, section as B
WHERE A.semester='Spring' AND A.year=2010 AND B.semester='Fall'
      AND B.year = '2009' AND A.course_id = B.course_id;
```



```
myuniversitydb=# SELECT course_id
                 FROM section as A, section as B
                 WHERE A.semester='Spring' AND A.year=2010 AND B.semester='Fall'
                       AND B.year = '2009' AND A.course_id = B.course_id;
ERROR:  column reference "course_id" is ambiguous
LINE 1: SELECT course_id
               ^
```

Select with Renaming

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Find all course ids taught in both Spring 2010 and Fall 2009.

```
SELECT A.course_id
FROM section as A, section as B
WHERE A.semester='Spring' AND A.year=2010 AND B.semester='Fall'
      AND B.year = '2009' AND A.course_id = B.course_id;
```



```
myuniversitydb=# SELECT A.course_id
                 FROM section as A, section as B
                 WHERE A.semester='Spring' AND A.year=2010 AND B.semester='Fall'
                       AND B.year = '2009' AND A.course_id = B.course_id;
 course_id
-----------
 CS-101
(1 row)
```

FROM with Renaming

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT *
FROM teacher AS mytable(a,b,c,d);
```



```
myuniversitydb=# SELECT *
myuniversitydb-# FROM teacher AS mytable(a,b,c,d);
   a   |     b      |     c      |    d
-------+------------+------------+----------
 10101 | Srinivasan | Comp. Sci. | 65000.00
 12121 | Wu         | Finance    | 90000.00
 15151 | Mozart     | Music      | 40000.00
 22222 | Einstein   | Physics    | 95000.00
 32343 | El Said    | History    | 60000.00
 33456 | Gold       | Physics    | 87000.00
 45565 | Katz       | Comp. Sci. | 75000.00
 58583 | Califieri  | History    | 62000.00
 76543 | Singh      | Finance    | 80000.00
 76766 | Crick      | Biology    | 72000.00
 83821 | Brandt     | Comp. Sci. | 92000.00
 98345 | Kim        | Elec. Eng. | 80000.00
(12 rows)
```

Number of attributes must be the same!

```
myuniversitydb=# SELECT *
myuniversitydb-# FROM teacher AS mytable(a,b,c,d,e);
ERROR:  table "mytable" has 4 columns available but 5 columns specified
```

FROM with Renaming

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT *
FROM teacher AS mytable(a,b,c);
```

```
myuniversitydb=# SELECT *
                 FROM teacher AS mytable(a,b,c);
   a   |     b      |     c      |  salary
-------+------------+------------+----------
 10101 | Srinivasan | Comp. Sci. | 65000.00
 12121 | Wu         | Finance    | 90000.00
 15151 | Mozart     | Music      | 40000.00
 22222 | Einstein   | Physics    | 95000.00
 32343 | El Said    | History    | 60000.00
 33456 | Gold       | Physics    | 87000.00
 45565 | Katz       | Comp. Sci. | 75000.00
 58583 | Califieri  | History    | 62000.00
 76543 | Singh      | Finance    | 80000.00
 76766 | Crick      | Biology    | 72000.00
 83821 | Brandt     | Comp. Sci. | 92000.00
 98345 | Kim        | Elec. Eng. | 80000.00
(12 rows)
```

Can re-name fewer attributes

```
myuniversitydb=# SELECT *
myuniversitydb-# FROM teacher AS mytable(a,b,,d);
ERROR:  syntax error at or near ","
LINE 2: FROM teacher AS mytable(a,b,,d);
                                   ^
```

FROM with Renaming

```
SELECT name, salary*1.1 AS abc, abc
FROM teacher
```

```
myuniversitydb=# SELECT name, salary*1.1 AS abc, abc
myuniversitydb-# FROM teacher;
ERROR:  column "abc" does not exist
LINE 1: SELECT name, salary*1.1 AS abc, abc
                                        ^
```

The new attribute name is not available to use in the same SELECT statement

GROUP BY with Renaming

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT SUBSTRING(name FROM 1 FOR 1) as abc, count(*)
FROM teacher
GROUP BY abc;
```

abc	*count*
B	1
C	2
W	1
M	1
E	2
K	2
G	1
S	2

```
SELECT SUBSTRING(name FROM 1 FOR 2) as abc, count(*)
FROM teacher
GROUP BY abc;
```

abc	*count*
Ca	1
Go	1
Sr	1
Ka	1
Cr	1
Ei	1
Ki	1
Si	1
El	1
Wu	1
Br	1
Mo	1

```
SELECT SUBSTRING(name FROM 2 FOR 1) as abc, count(*)
FROM teacher
GROUP BY abc;
```

abc	*count*
r	3
a	2
i	3
l	1
u	1
o	2

new idea

ORDER BY

Order By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the names of all teachers whose salary is greater than at least one teacher in the Comp. Sci. department.

the order of rows is *arbitrary*

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary
      AND B.dept_name = 'Comp. Sci.';
```

name
Einstein
Katz
Singh
Kim
Brandt
Wu
Crick
Gold

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary
    AND B.dept_name = 'Comp. Sci.'
ORDER BY name;
```

can also use A.name

name
Brandt
Crick
Einstein
Gold
Katz
Kim
Singh
Wu

Order By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the names of all teachers whose salary is greater than at least one teacher in Computer Science department.

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary AND B.dept_name = 'Comp. Sci.'
ORDER BY A.name;
```

name
Brandt
Crick
Einstein
Gold
Katz
Kim
Singh
Wu

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary AND B.dept_name = 'Comp. Sci.'
ORDER BY A.name DESC;
```

name
Wu
Singh
Kim
Katz
Gold
Einstein
Crick
Brandt

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary
      AND B.dept_name = 'Comp. Sci.'
ORDER BY B.name;
```

```
myuniversitydb # SELECT distinct A.name
                 FROM teacher AS A, teacher AS B
                 WHERE A.salary > B.salary
                     AND B.dept_name = 'Comp. Sci.'
                 ORDER BY B.name;
ERROR:  for SELECT DISTINCT, ORDER BY expressions must appear in select list
LINE 5:                  ORDER BY B.name;
                                  ^
```

Order By

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
SELECT *
FROM takes
ORDER BY semester ASC;
```

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
54321	CS-101	1	Fall	2009	A-
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
76543	CS-101	1	Fall	2009	A
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
98765	CS-101	1	Fall	2009	C-
12345	CS-347	1	Fall	2009	A
76653	EE-181	1	Spring	2009	C
98765	CS-315	1	Spring	2010	B
45678	CS-319	1	Spring	2010	B
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-319	2	Spring	2010	A
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

Order By

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
SELECT *
FROM takes
ORDER BY semester DESC;
```

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
98988	BIO-301	1	Summer	2010	
98988	BIO-101	1	Summer	2009	A
12345	CS-315	1	Spring	2010	A
45678	CS-319	1	Spring	2010	B
12345	CS-190	2	Spring	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-315	1	Spring	2010	B
12345	CS-101	1	Fall	2009	C
12345	CS-347	1	Fall	2009	A
98765	CS-101	1	Fall	2009	C-
76543	CS-101	1	Fall	2009	A
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
00128	CS-347	1	Fall	2009	A-
00128	CS-101	1	Fall	2009	A
54321	CS-101	1	Fall	2009	A-

Order By

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
SELECT *
FROM takes
ORDER BY semester ASC, sec_id DESC;
```


id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
45678	CS-101	1	Fall	2009	F
00128	CS-347	1	Fall	2009	A-
54321	CS-101	1	Fall	2009	A-
12345	CS-347	1	Fall	2009	A
76543	CS-101	1	Fall	2009	A
12345	CS-101	1	Fall	2009	C
98765	CS-101	1	Fall	2009	C-
44553	PHY-101	1	Fall	2009	B-
54321	CS-190	2	Spring	2009	B+
76543	CS-319	2	Spring	2010	A
12345	CS-190	2	Spring	2009	A
45678	CS-319	1	Spring	2010	B
12345	CS-315	1	Spring	2010	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
55739	MU-199	1	Spring	2010	A-
76653	EE-181	1	Spring	2009	C
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

first, sort increasing by **semester**.
second, *within* same semester,
sort decreasing by **sec_id**.

Order By

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
SELECT *
FROM takes
ORDER BY sec_id DESC, semester ASC;
```


id	*course_id*	*sec_id*	*semester*	*year*	*grade*
76543	CS-319	2	Spring	2010	A
12345	CS-190	2	Spring	2009	A
54321	CS-190	2	Spring	2009	B+
12345	CS-101	1	Fall	2009	C
98765	CS-101	1	Fall	2009	C-
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
00128	CS-347	1	Fall	2009	A-
54321	CS-101	1	Fall	2009	A-
00128	CS-101	1	Fall	2009	A
12345	CS-347	1	Fall	2009	A
76543	CS-101	1	Fall	2009	A
45678	CS-319	1	Spring	2010	B
12345	CS-315	1	Spring	2010	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
55739	MU-199	1	Spring	2010	A-
76653	EE-181	1	Spring	2009	C
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

first, sort decreasing by **sec_id**.
second, *within* same sec_id,
sort increasing by **semester**.

Order By

```
SELECT *
FROM takes
ORDER BY semester ASC, sec_id DESC;
```

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
45678	CS-101	1	Fall	2009	F
00128	CS-347	1	Fall	2009	A-
54321	CS-101	1	Fall	2009	A-
12345	CS-347	1	Fall	2009	A
76543	CS-101	1	Fall	2009	A
12345	CS-101	1	Fall	2009	C
98765	CS-101	1	Fall	2009	C-
44553	PHY-101	1	Fall	2009	B-
54321	CS-190	2	Spring	2009	B+
76543	CS-319	2	Spring	2010	A
12345	CS-190	2	Spring	2009	A
45678	CS-319	1	Spring	2010	B
12345	CS-315	1	Spring	2010	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
55739	MU-199	1	Spring	2010	A-
76653	EE-181	1	Spring	2009	C
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
SELECT *
FROM takes
ORDER BY semester ASC, sec_id DESC, year ASC;
```

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
54321	CS-101	1	Fall	2009	A-
12345	CS-347	1	Fall	2009	A
76543	CS-101	1	Fall	2009	A
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
00128	CS-347	1	Fall	2009	A-
00128	CS-101	1	Fall	2009	A
98765	CS-101	1	Fall	2009	C-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
54321	CS-190	2	Spring	2009	B+
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
45678	CS-319	1	Spring	2010	B
12345	CS-315	1	Spring	2010	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
45678	CS-101	1	Spring	2010	B+
55739	MU-199	1	Spring	2010	A-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

first, sort increasing by **semester**.
second, *within* same semester, sort decreasing by **sec_id**.
third, *within* same semester and sec_id, sort increasing by **year**.

Order By

```
SELECT *
FROM takes
ORDER BY semester ASC, sec_id DESC, year ASC;
```


id	*course_id*	*sec_id*	*semester*	*year*	*grade*
54321	**CS-101**	1	Fall	2009	**A-**
12345	**CS-347**	1	Fall	2009	**A**
76543	**CS-101**	1	Fall	2009	**A**
44553	**PHY-101**	1	Fall	2009	**B-**
45678	**CS-101**	1	Fall	2009	**F**
00128	**CS-347**	1	Fall	2009	**A-**
00128	**CS-101**	1	Fall	2009	**A**
98765	**CS-101**	1	Fall	2009	**C-**
12345	**CS-101**	1	Fall	2009	**C**
12345	**CS-190**	2	Spring	2009	**A**
54321	**CS-190**	2	Spring	2009	**B+**
76543	**CS-319**	2	Spring	2010	**A**
76653	**EE-181**	1	Spring	2009	**C**
45678	**CS-319**	1	Spring	2010	**B**
12345	**CS-315**	1	Spring	2010	**A**
19991	**HIS-351**	1	Spring	2010	**B**
23121	**FIN-201**	1	Spring	2010	**C+**
45678	**CS-101**	1	Spring	2010	**B+**
55739	**MU-199**	1	Spring	2010	**A-**
98765	**CS-315**	1	Spring	2010	**B**
98988	**BIO-101**	1	Summer	2009	**A**
98988	**BIO-301**	1	Summer	2010	

first, sort increasing by **semester**.
second, *within* same semester, sort decreasing by **sec_id**.
third, *within* same semester and sec_id, sort increasing by **year**.

Logical Order of Processing

FROM ⟶ WHERE ⟶ SELECT

↓ DISTINCT ?

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT

↓ ORDER BY ?

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT ⟶ ORDER BY

↓ AGGREGATIONS ?

FROM ⟶ WHERE ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT ⟶ ORDER BY

↓ GROUP BY ?

FROM ⟶ WHERE ⟶ GROUP BY ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT ⟶ ORDER BY

⟶ aggregations available from here onwards, each mini-table separately

---

new idea

IN

NOT IN

Set membership: IN

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the id, name, dept_name and salary of all teachers in Comp. Sci. or History department.

already seen

```
SELECT id, name, dept_name, salary
FROM teacher
WHERE dept_name = 'Comp. Sci.'
      OR dept_name = 'History';
```

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
32343	El Said	History	60000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
83821	Brandt	Comp. Sci.	92000.00

another way

```
SELECT id, name, dept_name, salary
FROM teacher
WHERE dept_name IN ('Comp. Sci.', 'History');
```

```
myuniversitydb=# SELECT id, name, dept_name, salary
                 FROM teacher
                 WHERE dept_name IN ('Comp. Sci.', 'History');
  id   |    name    | dept_name  |  salary
-------+------------+------------+----------
 10101 | Srinivasan | Comp. Sci. | 65000.00
 32343 | El Said    | History    | 60000.00
 45565 | Katz       | Comp. Sci. | 75000.00
 58583 | Califieri  | History    | 62000.00
 83821 | Brandt     | Comp. Sci. | 92000.00
(5 rows)
```

Set membership: NOT IN

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT id, name, dept_name, salary
FROM teacher
WHERE dept_name NOT IN ('Comp. Sci.', 'History');
```

id	*name*	*dept_name*	*salary*
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
98345	Kim	Elec. Eng.	80000.00

Set membership: IN

select all rows where this statement is true

```
SELECT id, name, salary
FROM teacher
WHERE dept_name IN ('Comp. Sci.', 'History');
```

an element

a list of elements of the same type

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

a tuple

```
SELECT course_id
FROM section
WHERE (semester, year) IN
        ( ('Summer', 2009), ('Fall', 2009) );
```

a list of tuples
(can also be a table)

course_id
BIO-101
CS-101
CS-347
PHY-101

an element can be a tuple!

new idea

HAVING

recall GROUP BY

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

GROUP BY sec_id, semester, year

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Step 1: a separate table for all rows with the same value on sec_id, semester, year

course_id	sec_id	semester	year	building	room_number	time_slot_id
CS-190	2	Spring	2009	Taylor	3128	A

course_id	sec_id	semester	year	building	room_number	time_slot_id
CS-101	1	Spring	2010	Packard	101	F
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
MU-199	1	Spring	2010	Packard	101	D
HIS-351	1	Spring	2010	Painter	514	C

course_id	sec_id	semester	year	building	room_number	time_slot_id
CS-190	1	Spring	2009	Taylor	3128	E
EE-181	1	Spring	2009	Taylor	3128	C

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-301	1	Summer	2010	Painter	514	A

course_id	sec_id	semester	year	building	room_number	time_slot_id
CS-101	1	Fall	2009	Packard	101	H
CS-347	1	Fall	2009	Taylor	3128	A
PHY-101	1	Fall	2009	Watson	100	A

course_id	sec_id	semester	year	building	room_number	time_slot_id
CS-319	2	Spring	2010	Taylor	3128	C

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B

Step 2: run the query separately on each table to get a single row per table

```
SELECT semester, year
```

semester	year
Spring	2009

semester	year
Spring	2009

semester	year
Fall	2009

semester	year
Summer	2010

semester	year
Summer	2009

semester	year
Spring	2010

semester	year
Spring	2010

recall GROUP BY

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

GROUP BY sec_id, semester, year

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Step 2: run the query separately on each table to get a single row per table

```
SELECT semester, year
```

semester	year
Spring	2009

semester	year
Spring	2009

semester	year
Fall	2009

semester	year
Summer	2010

semester	year
Summer	2009

semester	year
Spring	2010

semester	year
Spring	2010

Step 3: put the single row from each table together to get result

semester	year
Summer	2010
Spring	2010
Fall	2009
Spring	2009
Spring	2010
Spring	2009
Summer	2009

- different mini-tables can give same result
- we keep duplicates.

recall

GROUP BY: LOGICAL ORDER

① **FROM** table1, table2, ···

compute the input table from table1 × table2 × ···

② **WHERE** condition

choose rows in the input table that satisfy condition

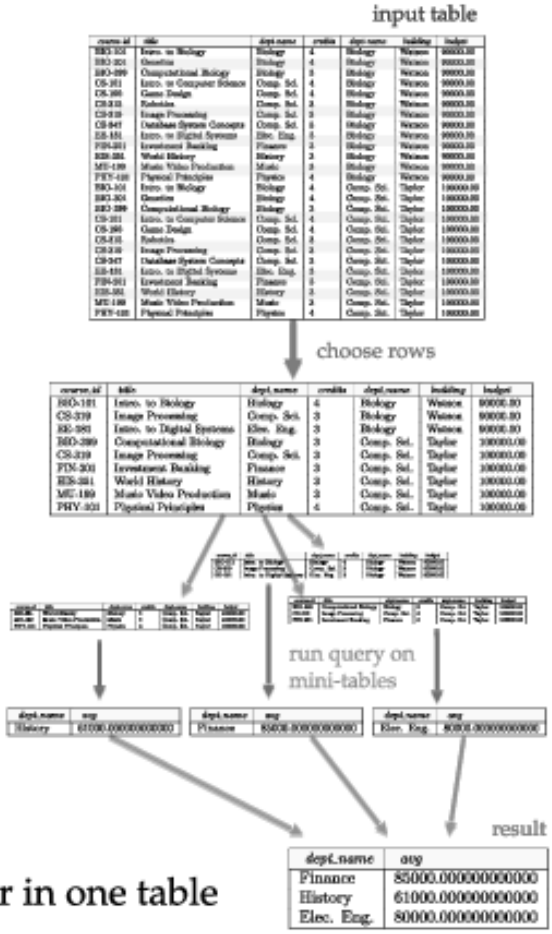
③ **GROUP BY** ···

break rows into mini-tables, with each mini-table having same values for the **GROUP BY** attributes

④ **SELECT** ···

run **SELECTION** statement *separately* on each mini table to get one output row per mini-table

the result is all the output rows put together in one table



dept.name	avg
Finance	85000.000000000000
History	61000.000000000000
Elec. Eng.	80000.000000000000

LOGICAL ORDER OF PROCESSING

FROM ⟶ **WHERE** ⟶ **SELECT**

↓ **DISTINCT** ?

FROM ⟶ **WHERE** ⟶ **SELECT** ⟶ **DISTINCT**

↓ **AGGREGATIONS** ?

FROM ⟶ **WHERE** ⟶ **AGGREGATIONS** ⟶ **SELECT** ⟶ **DISTINCT**

↓ **GROUP BY** ?

FROM ⟶ **WHERE** ⟶ **GROUP BY** ⟶ **AGGREGATIONS** ⟶ **SELECT** ⟶ **DISTINCT**

⟶ aggregations available from here onwards

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name, count(*)
FROM teacher
GROUP BY dept_name;
```

dept_name	*count*
Finance	2
History	2
Physics	2
Music	1
Comp. Sci.	3
Biology	1
Elec. Eng.	1

Find the department names and number of teachers in that department, **but only for those departments with at least two teachers.**

How to do this ?

dept_name	*count*
Finance	2
History	2
Physics	2
Comp. Sci.	3

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the department names and number of teachers in that department, but only for those departments with at least two teachers.

A first try that fails:

```
SELECT dept_name, count(*)
FROM teacher
WHERE count(*) >= 2
GROUP BY dept_name;
```

Why?

```
myuniversitydb=# SELECT dept name, count(*)
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE count(*) >= 2
myuniversitydb-# GROUP BY dept_name;
ERROR:  aggregate functions are not allowed in WHERE
LINE 3: WHERE count(*) >= 2
              ^
```

HAVING

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the department names and number of teachers in that department, but only for those departments with at least two teachers

```
⑤ SELECT dept_name, count(*)
① FROM teacher ④
② WHERE count(*) >= 2
③ GROUP BY dept_name;
```

① **FROM** table1, table2, ···

② **WHERE** condition

③ **GROUP BY** ···

④ **AGGREGATIONS** ···

⑤ **SELECT** ···

Problem: Groups and Aggregations not yet available, so cannot apply **count** to groups!

HAVING

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the department names and number of teachers in that department, but only for those departments with at least two teachers.


```
SELECT dept_name, count(*)
FROM teacher
GROUP BY dept_name
HAVING count(*) >= 2;
```

dept_name	*count*
Finance	2
History	2
Physics	2
Comp. Sci.	3

① **FROM** table1, table2, ···

② **WHERE** condition

③ **GROUP BY** ···

④ **AGGREGATIONS** ···

⑤ **SELECT** ···

⟸

HAVING:

executed *after* **GROUP BY** and **AGGREGATIONS**

applies to **whole groups** instead of single rows

each group that does not satisfy is **thrown away**

we *can* use aggregate functions inside HAVING

```
myuniversitydb=# SELECT dept_name, count(*)
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept_name
myuniversitydb-# HAVING count(*) >= 2;
 dept_name  | count
------------+-------
 Finance    |     2
 History    |     2
 Physics    |     2
 Comp. Sci. |     3
(4 rows)
```

EXAMPLE

For each course section offered in 2009, find the average total credits of all students enrolled in the section, but only for the sections with at least 2 students.

Long question! Take time to understand it.

```
SELECT course_id, sec_id, semester, year, avg(tot_cred)
FROM takes, student
WHERE year = 2009 and takes.id = student.id
GROUP BY course_id, sec_id, semester, year
HAVING count(student.id) >= 2;
```

student

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	



course_id	*sec_id*	*semester*	*year*	*avg*
CS-101	1	Fall	2009	65.0000000000000000
CS-190	2	Spring	2009	43.0000000000000000
CS-347	1	Fall	2009	67.0000000000000000

EXAMPLE

student

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
⑤ SELECT course_id, sec_id, semester, year, avg(tot_cred)
① FROM takes, student
② WHERE year = 2009 and takes.id = student.id
③ GROUP BY course_id, sec_id, semester, year
④ HAVING count(student.id) >= 2;
```

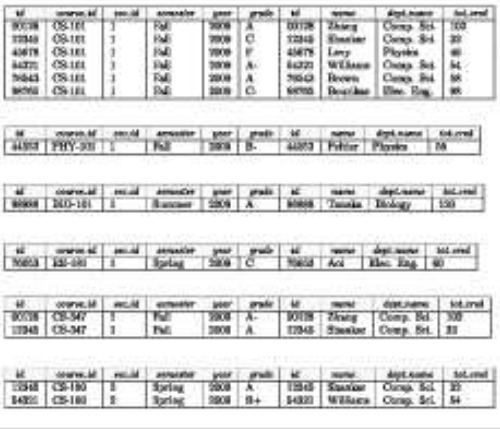
```
① FROM takes, student
② WHERE year = 2009 and takes.id = student.id
```

id	*course_id*	*sec_id*	*semester*	*year*	*grade*	*id*	*name*	*dept_name*	*tot_cred*
00128	CS-101	1	Fall	2009	A	00128	Zhang	Comp. Sci.	102
00128	CS-347	1	Fall	2009	A-	00128	Zhang	Comp. Sci.	102
12345	CS-101	1	Fall	2009	C	12345	Shankar	Comp. Sci.	32
12345	CS-190	2	Spring	2009	A	12345	Shankar	Comp. Sci.	32
12345	CS-347	1	Fall	2009	A	12345	Shankar	Comp. Sci.	32
44553	PHY-101	1	Fall	2009	B-	44553	Peltier	Physics	56
45678	CS-101	1	Fall	2009	F	45678	Levy	Physics	46
54321	CS-101	1	Fall	2009	A-	54321	Williams	Comp. Sci.	54
54321	CS-190	2	Spring	2009	B+	54321	Williams	Comp. Sci.	54
76543	CS-101	1	Fall	2009	A	76543	Brown	Comp. Sci.	58
76653	EE-181	1	Spring	2009	C	76653	Aoi	Elec. Eng.	60
98765	CS-101	1	Fall	2009	C-	98765	Bourikas	Elec. Eng.	98
98988	BIO-101	1	Summer	2009	A	98988	Tanaka	Biology	120


```
① FROM takes, student
② WHERE year = 2009 and takes.id = student.id
```

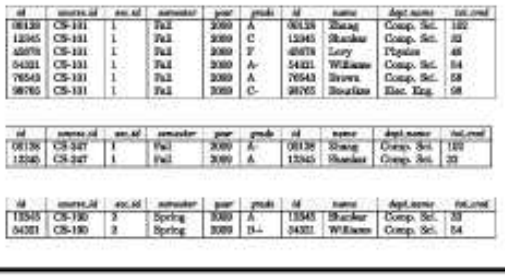

id	*course_id*	*sec_id*	*semester*	*year*	*grade*	*id*	*name*	*dept_name*	*tot_cred*
00128	CS-101	1	Fall	2009	A	00128	Zhang	Comp. Sci.	102
00128	CS-347	1	Fall	2009	A-	00128	Zhang	Comp. Sci.	102
12345	CS-101	1	Fall	2009	C	12345	Shankar	Comp. Sci.	32
12345	CS-190	2	Spring	2009	A	12345	Shankar	Comp. Sci.	32
12345	CS-347	1	Fall	2009	A	12345	Shankar	Comp. Sci.	32
44553	PHY-101	1	Fall	2009	B-	44553	Peltier	Physics	56
45678	CS-101	1	Fall	2009	F	45678	Levy	Physics	46
54321	CS-101	1	Fall	2009	A-	54321	Williams	Comp. Sci.	54
54321	CS-190	2	Spring	2009	B+	54321	Williams	Comp. Sci.	54
76543	CS-101	1	Fall	2009	A	76543	Brown	Comp. Sci.	58
76653	EE-181	1	Spring	2009	C	76653	Aoi	Elec. Eng.	60
98765	CS-101	1	Fall	2009	C-	98765	Bourikas	Elec. Eng.	98
98988	BIO-101	1	Summer	2009	A	98988	Tanaka	Biology	120

```
③ GROUP BY course_id, sec_id, semester, year
```




```
⑤SELECT course_id, sec_id, semester, year, avg(tot_cred)
①FROM takes, student
②WHERE year = 2009 and takes.id = student.id
③GROUP BY course_id, sec_id, semester, year
④HAVING count(student.id) >= 2;
```


```
④ HAVING count(student.id) >= 2
```



```
⑤ SELECT course_id, sec_id, semester, year, avg(tot_cred)
```

course_id	*sec_id*	*semester*	*year*	*avg*
CS-101	1	Fall	2009	65.000000000000000

course_id	*sec_id*	*semester*	*year*	*avg*
CS-347	1	Fall	2009	67.000000000000000

course_id	*sec_id*	*semester*	*year*	*avg*
CS-190	2	Spring	2009	43.000000000000000

```
myuniversitydb=# SELECT course_id, sec_id, semester, year, avg(tot_cred)
                 FROM takes, student
                 WHERE year = 2009 and takes.id = student.id
                 GROUP BY course id, sec id, semester, year
                 HAVING count(student.id) >= 2;
 course_id | sec_id | semester | year |         avg
-----------+--------+----------+------+---------------------
 CS-101    | 1      | Fall     | 2009 | 65.0000000000000000
 CS-190    | 2      | Spring   | 2009 | 43.0000000000000000
 CS-347    | 1      | Fall     | 2009 | 67.0000000000000000
(3 rows)
```

EXAMPLE

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

Find the names and ids of students who have taken only one class.

```
SELECT student.id, student.name
FROM takes, student
WHERE student.id = takes.id
GROUP BY student.id, student.name
HAVING count(*) = 1;
```

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

student

id	*name*
76653	Aoi
55739	Sanchez
19991	Brandt
44553	Peltier
23121	Chavez

Example

takes

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

Find the names, ids and number of courses, of students who have taken at least two courses.

```
SELECT student.id, student.name, COUNT(course_id)
FROM takes, student
WHERE student.id = takes.id
GROUP BY student.id, student.name
HAVING COUNT(course_id) >= 2;
```

id	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

student

id	name	count
76543	Brown	2
54321	Williams	2
45678	Levy	3
98765	Bourikas	2
00128	Zhang	2
12345	Shankar	4
98988	Tanaka	2

Example

takes

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

Find the names, ids and number of courses, of students who have taken at least two courses. List them in increasing number of courses.

```
SELECT student.id, student.name, COUNT(course_id)
FROM takes, student
WHERE student.id = takes.id
GROUP BY student.id, student.name
HAVING count(course_id) >= 2
ORDER BY count(course_id);
```

id	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

student

id	name	count
98765	Bourikas	2
54321	Williams	2
00128	Zhang	2
98988	Tanaka	2
76543	Brown	2
45678	Levy	3
12345	Shankar	4

HAVING without GROUP BY

student

id	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

HAVING without **GROUP BY** takes entire input table as one group

HAVING keeps or discards **entire** table

any condition must apply to entire table

```
SELECT tot_cred
FROM student
HAVING tot_cred > 50;
```



```
myuniversitydb=# SELECT tot_cred
myuniversitydb-# FROM student
myuniversitydb-# HAVING tot_cred > 50;
ERROR:  column "student.tot_cred" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT tot_cred
               ^
```

HAVING without GROUP BY

student

id	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

HAVING without **GROUP BY** takes entire input table as one group

all the attribute values must still be the same in **SELECT**

must output one line

. . . can't do much with it!

```
SELECT tot_cred
FROM student
HAVING tot_cred > 50;
```



```
myuniversitydb=# SELECT tot_cred
myuniversitydb-# FROM student
myuniversitydb-# HAVING tot_cred > 50;
ERROR:  column "student.tot_cred" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT tot_cred
               ^
```

```
SELECT avg(tot_cred)
FROM student
HAVING avg(tot_cred) > 50;
```



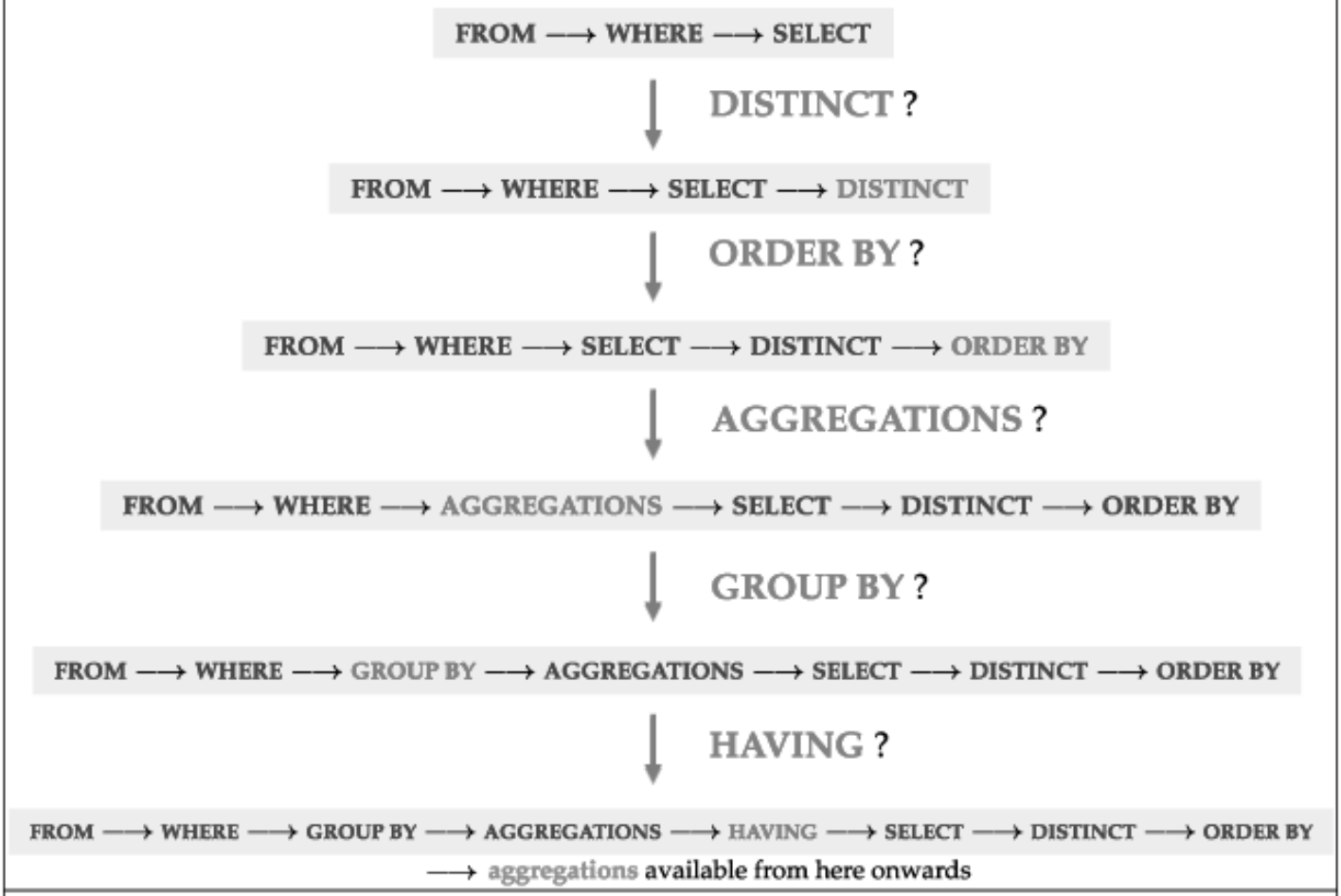
```
myuniversitydb=# SELECT avg(tot_cred)
myuniversitydb-# FROM student
myuniversitydb-# HAVING avg(tot_cred) > 50;
        avg
--------------------
 65.6923076923076923
(1 row)
```

```
SELECT avg(tot_cred)
FROM student
HAVING avg(tot_cred) > 500;
```



```
myuniversitydb=# SELECT avg(tot_cred)
                 FROM student
                 HAVING avg(tot_cred) > 500;
 avg
-----
(0 rows)
```

Logical Order of Processing



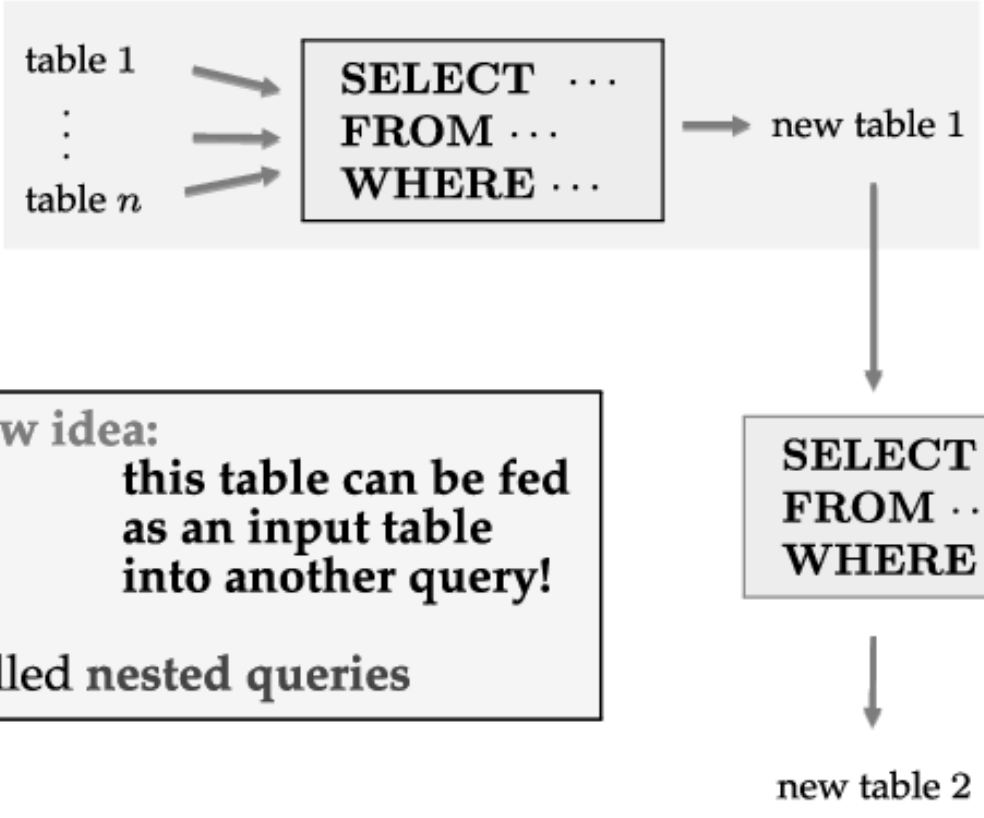
new idea

NESTED QUERIES

Nested Queries

So far:

output of a query is a single table



New idea:

this table can be fed as an input table into another query!

Called **nested queries**

Nested Queries

You can use a nested subquery in:

FROM : to specify a new input table

WHERE : condition to select rows (e.g., using IN)

SELECT : add a new attribute to output table

HAVING : decide which mini-tables to throw away

Subqueries should **always** be inside ( )

Subqueries are often slow and inefficient

new idea

NESTED QUERIES

FROM

Nested Queries: From

use in **FROM**

SELECT ···
FROM (specify table(s) here)
WHERE ···

can be generated from a query!

SELECT ···
FROM (**SELECT** ···
FROM ···
WHERE ···)
WHERE ···

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the departments where the average teacher salary is greater than 42, 000

So far: use HAVING

```
SELECT dept_name, avg(salary) as avg_salary
FROM teacher
GROUP BY dept_name
HAVING avg(salary) >= 42000;
```



dept_name	*avg_salary*
Finance	85000.000000000000
History	61000.000000000000
Physics	91000.000000000000
Comp. Sci.	77333.333333333333
Biology	72000.000000000000
Elec. Eng.	80000.000000000000

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the departments where the average teacher salary is greater than 42, 000

Step 1: construct the table with avg(salary) for each dept_name

```
SELECT dept_name, avg(salary) as avg_salary
FROM teacher
GROUP BY dept_name
```



dept_name	*avg_salary*
Finance	85000.000000000000
History	61000.000000000000
Physics	91000.000000000000
Music	40000.000000000000
Comp. Sci.	77333.333333333333
Biology	72000.000000000000
Elec. Eng.	80000.000000000000

mytable

Step 2: use this table (call it mytable) as an input table to a query that chooses the rows with avg_salary > 42, 000

```
SELECT dept_name, avg_salary
FROM mytable
WHERE avg_salary > 42000
```

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the departments where the average teacher salary is greater than 42, 000

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, avg(salary) as avg_salary
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE avg_salary > 42000;
```

must give table a name

SELECT dept_name, **avg**(salary) **as** avg_salary
FROM teacher
GROUP BY dept_name

mytable

+

SELECT dept_name, avg_salary
FROM mytable
WHERE avg_salary > 42000

dept_name	*avg_salary*
Finance	85000.000000000000
History	61000.000000000000
Physics	91000.000000000000
Comp. Sci.	77333.333333333333
Biology	72000.000000000000
Elec. Eng.	80000.000000000000

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the departments where the average teacher salary is greater than 42, 000

```
SELECT dept_name, avg(salary)
FROM (SELECT dept_name, avg(salary)
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE avg(salary) > 42000;
```

```
myuniversitydb=# SELECT dept name, avg(salary)
                 FROM (SELECT dept_name, avg(salary)
                       FROM teacher
                       GROUP BY dept_name) as mytable
                 WHERE avg(salary) > 42000;
ERROR:  column "salary" does not exist
LINE 1: SELECT dept_name, avg(salary)
                                ^
```

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the departments where the average teacher salary is greater than 42, 000

```
SELECT dept_name, avg(salary)
FROM (SELECT dept_name, avg(salary)
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE avg(salary) > 42000;
```

✗

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, avg(salary) as avg_salary
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE avg_salary > 42000;
```

✓

Nested Queries: From

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Find the **maximum** total number of credits over all departments.

```
SELECT max(summed_credits)
FROM (SELECT dept_name, sum(credits)
                        as summed_credits
      FROM course
      GROUP BY dept_name) as mytable;
```

SELECT dept_name, sum(credits)
as summed_credits
FROM course
GROUP BY dept_name

\+ **mytable**

SELECT max(summed_credits)
FROM mytable

dept_name	*summed_credits*
Finance	3
History	3
Physics	4
Music	3
Comp. Sci.	17
Biology	11
Elec. Eng.	3

max
17

Nested Queries: From

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the department names and number of teachers in that department, but only for those departments with at least two teachers

```
SELECT dept_name, count(*) as count_teachers
FROM teacher
GROUP BY dept_name
HAVING count(*) >= 2;
```

```
SELECT dept_name, count_teachers
FROM (SELECT dept_name, count(*) as count_teachers
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE count_teachers >= 2;
```

dept_name	*count_teachers*
Finance	2
History	2
Physics	2
Comp. Sci.	3

Can implement HAVING using nested queries!

new idea

NESTED QUERIES

WHERE

Nested Queries: Where

use in **WHERE**

SELECT attribute 1, attribute 2, ···
FROM ···
WHERE (attribute 3, attribute 4) **IN**
(specify a list/table here)

can be generated from a query!

SELECT attribute 1, attribute 2, ···
FROM ···
WHERE (attribute 3, attribute 4) **IN**
(**SELECT** attribute 5, attribute 6
FROM ···
WHERE ···)

domains must be the same!

Using **IN** is one way.
Can use **any** condition that returns a Boolean.

Nested Queries: Where

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Find all courses taught in both Spring 2010 and Fall 2009.

```
SELECT course_id
FROM section
WHERE course_id IN
        (SELECT course_id
         FROM section
         WHERE semester='Spring' AND year=2010)
      AND semester='Fall' AND year = '2009';
```

course_id
CS-101

course_id
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199

Nested Queries: Where

number and domains must be the same!

```
SELECT course_id
FROM section
WHERE course_id IN
      (SELECT course_id, building
       FROM section
       WHERE semester='Spring' and year=2010);
```



```
ERROR:  subquery has too many columns
LINE 3: WHERE course_id IN
                         ^
```

Nested Queries: Where

takes

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

teaches

id	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Find the total number of distinct students who have taken sections taught by the teacher with id 10101

```
SELECT count (distinct id)
FROM takes
WHERE (course_id, sec_id, semester, year) IN
          (SELECT course_id, sec_id, semester, year
           FROM teaches
           WHERE teaches.id=10101);
```

course_id	sec_id	semester	year
CS-101	1	Fall	2009
CS-315	1	Spring	2010
CS-347	1	Fall	2009

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
45678	CS-101	1	Fall	2009	F
54321	CS-101	1	Fall	2009	A-
76543	CS-101	1	Fall	2009	A
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B

count
6

Nested Queries: Where

takes

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

teaches

id	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Find the total number of distinct students who have taken sections taught by the teacher with id 10101

```
SELECT count (distinct id)
FROM takes
WHERE (course_id, sec_id, semester, year) IN
          (SELECT course_id, sec_id, semester, year
           FROM teaches
           WHERE teaches.id=10101);
```

count
6

```
SELECT count (distinct id)
FROM takes
WHERE (course_id, sec_id, semester, year) IN
     ( ('CS-101', '1', 'Fall', 2009),
       ('CS-315', '1', 'Spring', 2010),
       ('CS-347', '1', 'Fall', 2009) );
```

Nested Queries: Where

department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Find all departments with the maximum budget.

```
SELECT dept_name
FROM department
WHERE department.budget IN
    (SELECT max(budget)
     FROM department);
```

dept_name
Finance

Scalar subquery: returns table with a single value.
whole table can be treated as a single value

```
SELECT dept_name
FROM department
WHERE department.budget =
          (SELECT max(budget)
          FROM department);
```

new idea

NESTED QUERIES

SELECT

Nested Queries: Select

use in **SELECT**

can be generated from a query!

```
SELECT attribute 1, attribute 2, ···
FROM ···
WHERE ···
```

must return a single row with a single attribute

```
SELECT attribute 1, (SELECT attribute 2 ···), ···
FROM ···
WHERE ···
```

Nested Queries: Select

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

List all teachers together with the total number of other teachers in that teacher's department.

```
SELECT name, (SELECT count(*)
              FROM teacher AS B
              WHERE B.dept_name = A.dept_name)
FROM teacher AS A;
```

run 12 times
once for each output line

name	*count*
Srinivasan	3
Wu	2
Mozart	1
Einstein	2
El Said	2
Gold	2
Katz	3
Califieri	2
Singh	2
Crick	1
Brandt	3
Kim	1

Nested Queries: Select

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Give the name of each teacher and the rank of that teacher's salary

```
SELECT name, (SELECT count(*)
              FROM teacher AS B
              WHERE B.salary >= A.salary)
FROM teacher AS A;
```

run 12 times

name	*count*
Srinivasan	9
Wu	3
Mozart	12
Einstein	1
El Said	11
Gold	4
Katz	7
Califieri	10
Singh	6
Crick	8
Brandt	2
Kim	6

ties share the **last** position of their group;
for the usual rank use `B.salary > A.salary` and add 1

new idea

NESTED QUERIES

HAVING

Nested Queries: Having

use in **HAVING**

```
SELECT attribute 1, attribute 2, ···
FROM ···
WHERE ···
GROUP BY ···
HAVING condition
```



can use a subquery!

Nested Queries: Having

Find the department names and number of teachers in that department, but only for those departments with at least two teachers.

```
SELECT dept_name, count(*)
FROM teacher
GROUP BY dept_name
HAVING count(*) >= 2;
```

→

dept_name	*count*
Finance	2
History	2
Physics	2
Comp. Sci.	3

Find the department names and number of teachers in that department, but only for those departments with **maximum number of teachers.**

```
SELECT A.dept_name, count(*)
FROM teacher as A
GROUP BY A.dept_name
HAVING count(*) >=
        (SELECT max(c)
          FROM (SELECT count(*) as c
                FROM teacher as B
                GROUP BY B.dept_name)
        );
```

Scalar subquery

→

dept_name	*count*
Comp. Sci.	3

Renaming Rules

```
SELECT id
FROM (SELECT student.id, teacher.id
      FROM student, teacher) as T;
```

```
SELECT student.id
FROM (SELECT student.id, teacher.id
      FROM student, teacher) as T;
```

```
SELECT T.student.id
FROM (SELECT student.id, teacher.id
      FROM student, teacher) as T;
```

once the subquery is named `T`, the old names (`student`, `teacher`) are **no longer visible**

```
SELECT T.a
FROM (SELECT student.id as a, teacher.id as b
      FROM student, teacher) as T;
```

Scoping Rules

Tables available :

defined in that subquery
defined in the query that **contains** that subquery

```
SELECT mytable.dept_name, mytable.avg_salary
FROM (SELECT dept_name, avg(salary) as avg_salary
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE mytable.avg_salary >= 42000;
```

dept_name	*avg_salary*
Physics	91000.000000000000
Biology	72000.000000000000
Elec. Eng.	80000.000000000000
Finance	85000.000000000000
Comp. Sci.	77333.333333333333
History	61000.000000000000

```
SELECT mytable.dept_name, mytable.avg_salary
FROM (SELECT dept_name, avg(salary) as avg_salary
      FROM teacher
      GROUP BY dept_name) as mytable
WHERE mytable.avg_salary >=
       (SELECT max(B.avg_salary)
        FROM mytable as B);
```

```
ERROR:  relation "mytable" does not exist
LINE 7:         FROM mytable as B);
                     ^
```

if same table name globally and locally : local name takes precedence

Nested Queries

You can use a nested subquery in:

FROM : to specify a new input table

WHERE : condition to select rows (e.g., using IN)

SELECT : add a new attribute to output table

HAVING : decide which mini-tables to throw away

Subqueries should always be inside ( )

Subqueries are often inefficient; should prefer other ways

Logical Order of Processing

